

The One Introduction Problem: Consent-First Decision Support for Vouchsafe

Sequential reciprocal matching under uncertainty

CRSU x Romeo & Juliet Hackathon · Vouchsafe-focused challenge

Final participant specification 1.0.0 · 5 October 2026

Build a decision policy that chooses who should be introduced to whom, when an introduction should happen, and when further information is worth asking for. Your policy will operate across a changing population of synthetic people, with incomplete observations, reciprocal preferences and feedback that arrives after decisions have been made.

The deliverable is a reproducible Python system that interacts with the supplied simulator, together with a technical report explaining its assumptions, experiments and limitations. A website or mobile interface is optional. It does not replace the decision policy.

This release freezes the participant data, policy interface, outcome definition, technical ranking and submission rules. The complete kit runs offline with Python 3.10 or later. The technical rules in this document govern this repository release.

Product relevance: a useful Vouchsafe prototype

This challenge models one narrow component that could support a Vouchsafe introduction workflow: choosing whether and when to suggest a mutually eligible introduction as information arrives over time. Treat the policy as decision support. A person or authorised operator remains responsible for approving any real-world action; the competition policy must not contact people, make commitments, or override consent.

Alongside the executable policy and technical report, include a concise product integration note in the report. Explain how a real service could preserve explicit consent and hard constraints, show a human-readable reason and uncertainty for each suggestion, allow a person to decline or correct information, minimise data sent to the policy, and monitor failures or uneven outcomes. Identify which assumptions in this simulator would need validation before any real use. The integration note is part of the report and does not change the published technical score.

The supplied simulator is synthetic research material, not a model of actual Vouchsafe members or a validated predictor of relationship outcomes. A good score is evidence only about the supplied simulator.

1 The challenge

An introduction involves two independent people. Both must be willing to consider it, both must be able to act on it, and neither person's constraints can be overridden by a favourable score.

The system also has to make choices across the whole pool. Introducing one person to somebody today changes the options available to everybody else. Waiting may allow a better opportunity to appear, but it can also leave people without an introduction. Asking another question may resolve a genuine uncertainty, or consume a limited budget without changing the decision.

Your task is to design a policy that manages these trade-offs over time. At each step, it must decide which information to request and which feasible, non-overlapping introductions to propose. It may choose to make no introduction.

The setting is inspired by Romeo & Juliet's introduction process. Every person and outcome in the competition environment is synthetic. The challenge concerns decision-making under uncertainty; simulator performance is not evidence that a model can predict a relationship.

2 Why independent pair ranking is insufficient

Consider four available people. Suppose a policy assigns the following illustrative expected-outcome values to four feasible pairs.

Pair	Illustrative value
A and B	0.90
C and D	0.05
A and D	0.65
B and C	0.65

Selecting the highest individual pair first gives A-B and C-D, with a total value of 0.95. Selecting A-D and B-C gives 1.30. These numbers illustrate an allocation problem; they are not compatibility percentages or measured results.

The sequential problem adds further questions. What if B has not answered a decisive question? What if D arrives tomorrow? What if A has already received an introduction and is waiting for a response? A useful policy must connect its estimates to these operational constraints.

3 What your system must do

Your system must perform the following tasks through the participant interface.

- Read the currently observable population and feedback history.
- Distinguish confirmed constraints from missing or declined information.
- Select clarification requests within the daily budget.
- Assess both directions of a potential introduction.
- Allocate feasible pairs across the pool, using each available person at most once in a batch.
- Update its decisions as people arrive, become unavailable or receive feedback.
- Produce reproducible outputs and enough evidence to explain its decisions.

You may use statistical models, graph optimisation, bandit methods, reinforcement learning, rules or a combination. A simpler method with convincing experiments is a valid submission. No particular model family is required.

The executable output consists of clarification requests and proposed pairs through the two-phase JSON protocol; either list may be empty. Probability estimates are optional and do not affect ranking. If reported, distinguish A accepting B, B accepting A, and both accepting, and define the prediction target and observation window. These are different quantities. Multiplying directional probabilities requires an independence assumption that may not hold.

4 The development dataset

The participant package contains 2,000 synthetic adults across ten independent pools of 200 people. Each supplied snapshot represents the information available at simulation day 30. Across the snapshots there are 613 simulated introductions, 1,270 observable feedback events and 1,129 short synthetic conversation records.

File	Purpose
state.json	Complete observable state for a pool at the snapshot time
members.jsonl	Structured member observations and missingness states
questionnaires.jsonl	Synthetic answers in a questionnaire-shaped representation
conversations.jsonl	Short authored conversation fragments linked to synthetic members
introductions.jsonl	Historical assignments with endpoints and response deadlines
feedback.jsonl	Feedback observable by the snapshot time
members_flat.csv	Simplified view for initial exploration
data_manifest.json	Pool membership, development seeds and dataset splits

Six pools are designated for training, two for validation and two for development testing. Keep those populations separate when comparing methods. Do not randomly split rows from the same people or pairs across training and testing.

The static files support exploration and current-state decisions. The simulator supports repeated episodes from day zero, so teams can collect their own decision histories under different policies. Use the full JSON contract for implementation; the flattened CSV omits important constraint and timing information.

The records contain no real member profiles, photographs, contact details, audio or copied conversations. They preserve useful structural features of an introduction workflow, but do not reproduce a validated distribution of real relationship outcomes. Conversation fragments are authored templates. They are suitable for testing input handling, not for benchmarking realistic speech recognition or subtle psychological interpretation.

5 What the policy can observe

Each member has an opaque synthetic identifier, a pool identifier, a synthetic age, an explicitly stated gender, a fictional geographic zone, an arrival day and current availability. Observations also include reciprocal preferences, practical constraints and softer self-reported information.

Information	Examples	How to use it
Reciprocal eligibility	Acceptable ages and explicitly stated genders to meet	Check both directions
Relationship constraints	Relationship structure, smoking preferences and children-related constraints	Enforce the supplied hard rules
Practical feasibility	Acceptable zones and overlapping schedule slots	Establish whether an introduction is currently feasible

Information	Examples	How to use it
Softer observations	Relationship goal, pace, lifestyle, conversation preferences and available time	Inform estimates without overriding constraints
Observation history	Missingness status and day an answer became observable	Prevent future information entering earlier decisions
Interaction history	Assignments and feedback received so far	Support sequential learning and avoid repeated introductions

Every questionnaire field has a value, a status and an observation time. A null value is unknown. It is never shorthand for zero, rejection or lack of interest. Status distinguishes an observed answer from a question not yet asked and an answer the person declined to disclose.

Age and gender do not imply an unstated preference. Use the supplied `who_to_meet` field. The fictional zones are allocation constraints, not real locations or travel distances. Relocation interest does not automatically override a person's currently acceptable zones.

The versioned `docs/DATA_CONTRACT.md` in the participant package defines the complete field vocabulary and missing-value semantics.

6 Decisions and simulator mechanics

Each simulation step represents one day. The release settings are as follows.

Setting	Value
Decision horizon used by the supplied demonstration	60 days
Follow-up after the decision horizon	40 days with no new introductions
Clarification budget	12 units per day
Clarify the hard-constraint bundle for one member	3 units
Clarify one named soft field	1 unit
Introduction response deadline	7 days after assignment
Concurrent introductions	At most one per person
Repeated pair	Not permitted within an episode

The hard-constraint bundle is a simulator abstraction. Its price does not represent the burden of asking every corresponding question in a real service. Clarification is immediate in v1.0 and returns exact simulated self-reports when disclosed. Declined answers remain unavailable.

A policy follows this cycle:

```

state = simulator.observe()
asks = policy.ask(state)
ask_results = simulator.resolve_asks(asks)
state = simulator.observe()
pairs = policy.match(state, ask_results)
simulator.advance(pairs)
feedback = simulator.receive_feedback()

```

Refresh the observation after clarification so that matching uses the answers just received. The policy may retain its own memory between steps within an episode. Independent evaluation episodes must start from the declared initial policy state, with no information carried from private evaluation runs.

Waiting is represented by omitting a person from the proposed pairs. An empty list is valid. The simulator releases feedback when its observation time is reached; no future outcome is included in the current state.

7 Valid introductions

The supplied eligibility function distinguishes feasible pairs, infeasible pairs and pairs that need clarification. A known hard failure takes precedence over missing information. An unresolved hard constraint blocks an introduction until the required information is available.

For a proposed pair to be valid, all of the following must hold:

- Both identifiers refer to distinct, available adults in the same pool.
- Each person is within the other's stated acceptable age range and gender preferences.
- Their relationship structures satisfy the development contract.
- Neither person's smoking or children-related constraints are violated.
- Each accepts the other's current geographic zone and their schedules overlap.
- The pair has not already been introduced in the episode.
- Neither person appears in another pair in the same batch or has an outstanding introduction.

The simulator validates the complete batch before applying it. Invalid actions raise an error and do not create a partial batch. Any invalid action, malformed output, runtime failure or exceeded execution limit makes that evaluation episode invalid. A submission with an invalid assessed episode is ineligible for technical ranking. Teams may replace their submitted version before the deadline; no participant fixes are accepted after it, including when private assessment happens later. See Section 12 for validation and the version-freeze policy.

A high predicted outcome cannot compensate for a constraint violation. A person without a feasible candidate may remain unmatched. The report should explain such cases rather than silently remove them from coverage statistics.

8 Feedback and the outcome of interest

Assignments, acceptance, dates and interest in meeting again are separate events. Your system must preserve that distinction.

Event	Meaning
Introduction assigned	The policy selected a valid pair
Directional response	One person answered Yes or No, or did not respond by the deadline

Event	Meaning
Mutual acceptance	Both people returned Yes to the introduction
Date happened	The simulator recorded a meeting after mutual acceptance
Second-meeting intention	Each person independently reported whether they wanted to meet again

The principal outcome is Mutual Second-Meeting Intention. An introduction meets this definition only when a first date happens within 30 simulation days of assignment and both people return Yes within three simulation days of that date.

The development simulator uses whole days. The three-day feedback window approximates a 72-hour window; it is not an exact timestamp implementation. A late Yes is retained in the feedback history but does not satisfy this outcome definition.

A missing response remains missing. For a prediction target such as "recorded Yes by the deadline", no response means that the specified event did not occur, but it does not establish dislike or rejection. Similarly, a pair that never dates has no observed second-meeting preference.

At any snapshot, recent introductions may still be awaiting outcomes. This is right-censoring: the observation window has not finished. Do not label every unresolved introduction as unsuccessful. Evaluate mature outcomes after the prescribed follow-up period.

9 Learning from the data

Feedback is selective because the system observes outcomes for the pairs it chooses to introduce. It does not receive ground-truth labels for every possible pair. A policy therefore influences both the outcomes and the evidence available for its next decisions.

Your report should address how you handle cold starts, uneven history, unknown preferences and delayed observations. If your method relies on exploration, explain what is being explored and how hard constraints remain protected.

Use only evidence available at the time of each action. A day-30 questionnaire answer may have been learned after an introduction made on day 8. The snapshots include field observation times and clarification logs; for model training, retain pre-action observations from simulator rollouts rather than attaching final profiles to earlier outcomes.

Historical logging propensities are not supplied. The propensity field is null. Do not claim unbiased inverse-propensity evaluation from these files. Compare complete policies in controlled simulator episodes, and explain any additional assumptions behind offline analyses.

Separate model uncertainty from missing data and stochastic outcomes. An explanation must identify observed evidence and unresolved information without inventing a personal history or presenting a latent simulation value as an observation.

10 Evaluation and ranking

Each assessed episode has 200 members, 60 decision days and 40 follow-up days. There are no new asks or introductions during follow-up. Feedback remains observable. Every episode begins with empty policy memory. The denominator is the number of members who arrived by decision day 59, including members who later left or were never served.

For each episode, let N be the number of members who arrived by decision day 59 and M the number of introductions that qualify for Mutual Second-Meeting Intention:

$\text{MSMI per 100 arrived members} = 100 \times M / N$

One qualifying pair counts as one MSMI outcome, not two. For example, one qualifying introduction among 200 arrived members scores 0.5. Coverage is the proportion of those same N members who received at least one introduction: $\text{distinct arrived members introduced} / N$. Count each served member once, even if they received several introductions. Include members who later left or paused in both denominators. If N is zero, both metrics are defined as zero.

The primary score is the equally weighted mean of MSMI per 100 arrived members across six scenario families: standard, sparse geography, cold start, delayed dates, shifted outcome weights and changing response conditions. Each family uses 20 independent private seeds, giving 120 episodes. Within each family, average episode scores; then average the six family means. Do not pool denominators across families or select a team's best seed. Private seeds and worlds are not released to policies. The public variants let teams test the same categories without exposing assessed episodes.

Rank eligible submissions by this primary score in descending order. Break exact unrounded ties by mean distinct-member introduction coverage, then mean mutual acceptances per 100 arrived members, then lower mean clarification cost, then lower measured inference time. A remaining exact tie shares rank. There is no subjective multiplier or undisclosed scoring weight. All assessed episodes must be valid. The supplied evaluator writes the full episode results and ranking statistics; organisers run the same protocol against held-back worlds using isolated policy containers.

Report assignments, mutual acceptances, dates, MSMI, coverage, clarification cost and missing feedback separately. Report first-introduction waiting times, including the unserved count. Include results across multiple seeds and each scenario, rather than only a favourable run. A zero MSMI count can occur in a small synthetic episode; it is not evidence that two methods are equivalent. Give uncertainty and failure analysis where the number of observed outcomes is small.

Compare against the supplied greedy, no-clarification and random-feasible baselines using identical seeds and variants. The command-line evaluator supports all three. At least one hypothesis-driven ablation is required, such as disabling clarification or replacing global allocation with greedy ranking. Probability estimates are optional and do not affect ranking; if reported, define the target and show calibration on held-out observations. Missing responses are not preference labels.

11 Execution rules

Implement the JSON stdin/stdout protocol in docs/POLICY_INTERFACE.md. Each invocation reads one request, writes one JSON response and exits. Memory is passed explicitly in the response and returned in the next request. Use stdout only for protocol JSON; send diagnostics to stderr. Your policy must handle an empty population and an empty feasible graph.

Assessed inference runs offline on CPU only, with 2 CPU cores, 1 GiB memory, 64 processes and a 10-second wall-clock limit per invocation, including process startup. The request, response and carried memory each have a 1 MiB limit. A policy image may contain at most 2 GiB of required inference assets. Persistent filesystem state between invocations is not available. Network calls, external paid services and GPU requirements are not permitted during evaluation. Training may use other compute; document it and include all assets required for offline inference.

Package a Docker image using a committed Dockerfile. The starter Dockerfile is immediately runnable. Pin any added dependencies and include their licences. The local subprocess runner

is for trusted development code only; it is not a security sandbox. Container mode isolates policy filesystem access from host evaluation files, disables network access and enforces the stated container limits. Organisers must use container mode for assessed third-party submissions and keep private worlds outside the policy image.

Only observable state, clarification results, feedback, policy-owned memory and declared training assets are permitted inputs. Do not read hidden simulator objects, reconstruct hidden answers from public generator seeds or IDs, inspect organiser files, or exploit future outcomes. The inspectable public simulator is an experimental tool, not an oracle for the policy. Deliberate bypass of the observation contract disqualifies a submission regardless of score.

12 Schedule and submission

Milestone	Date and time (IST)
Round 1 research submission closes	9 October 2026, 23:59
Round 1 results announced by	11 October 2026, 22:00
Round 2 build starts	12 October 2026
Final build submission closes	18 October 2026, 23:59

All times are Indian Standard Time (UTC+05:30). The research deadline is end of day on 9 October, equivalent to 18:29 UTC. Round 1 results will be announced by 10 PM IST on 11 October. Round 2 begins on 12 October; its final submission deadline remains 18 October at 23:59 IST (18:29 UTC). Teams have one to three participants. Mentorship and workshops are not included in this CRSU track. Each team may submit one entry per round; revisions received before that round's deadline replace its earlier entry. Research assessment does not change the public technical rules.

Round 1 research submissions use a Google Form. The organisers will release the form link soon and announce it through [the official Discord server](#) and this repository's submission guide. The form link is not yet available in this release. GitHub Issues are not the Round 1 submission route.

Prepare the research note as a PDF or Markdown document. It must explain the hypothesis, reciprocal feasibility, any optional probability estimates, allocation method, clarification strategy, baseline, planned experiments and expected failure cases. Follow the form's upload or document-link instructions once released. The Google Form receipt timestamp determines whether the research submission or revision arrived before the deadline. Retain the form's submission confirmation. If supplying a document link, pin a fixed version rather than changing the linked document after the deadline.

Round 2 final build submissions use this repository's Final submission issue template. These are public submissions: include your team name and public project links, not phone numbers, personal dating information, passwords or private participant data. The latest version pinned in the issue before the final deadline determines the assessed build; an earlier issue creation time does not make a later revision timely. Publish a source archive or repository accessible without requesting credentials. If using a repository, provide its full 40-character commit SHA. Alternatively, provide an immutable public archive URL and the full SHA-256 checksum of that archive's exact bytes. A latest-branch ZIP URL, editable shared-drive file or mutable download link is not an immutable archive. A changed archive or checksum mismatch is invalid; organisers assess only the version pinned before the deadline. Results generated after the deadline do not change the assessed version.

The final project must contain the policy, Dockerfile, pinned dependencies, inference assets, reproducible evaluation command, machine-readable results, technical report and attribution. Include all three baseline comparisons, at least one ablation, scenario-level results, declared training seeds and inference random seed. Explain failures involving sparse supply, withheld answers, delayed feedback and competition for the same candidate. A dashboard or presentation is optional; the runnable policy and report are required. docs/SUBMISSION.md provides the complete checklist and a copyable local verification sequence.

For Round 2, teams can obtain validation results before the final deadline by running the published tests, data verification and local/container evaluation commands in docs/SUBMISSION.md. A successful public run checks the interface and tested episodes; it does not guarantee that every private episode will be valid. Organiser validation feedback, if received before the deadline, may be used to revise and resubmit before it. Individual pre-deadline organiser feedback and early private assessment are not guaranteed.

The Round 1 research note is frozen at the Google Form deadline. The Round 2 submitted commit or archive is frozen at the final build deadline. Private assessment may take place afterwards. Failures first discovered during that assessment do not permit post-deadline code, dependency, asset or packaging fixes, and an invalid assessed episode makes the entry ineligible. Organiser errors or defects in the common kit must be handled through the public corrections policy for all affected teams, without silently replacing a team's submitted version.

13 Communication and corrections

Use the Question template in this repository's Issues tab for technical questions. Search existing questions first and keep one topic per issue. Maintainers answer publicly so every team receives the same clarification. Use the Bug report template for reproducible kit defects. Never attach real-member data or credentials.

The repository is the authoritative place for the PS, data, FAQ, version history and technical clarifications. Official event announcements and reminders are shared through [this event's Discord server](#). Discord, email announcements and the Unstop listing should link to this repository. No separate chat account is needed to obtain the kit. Rule-changing corrections must be written in CHANGELOG.md and shared through a public issue before they apply; informal messages do not silently amend this release.

14 Data provenance and permitted use

Every identifier, adult profile, preference, questionnaire answer, conversation fragment, introduction and outcome is newly generated or authored synthetic content. No real Supabase rows, names, messages, contact details, photos or audio are distributed. Schema patterns informed the dataset design; demographic proportions and outcome behaviour are invented assumptions rather than fitted or validated real-member distributions.

There are 2,000 adults in ten disjoint pools, 613 introductions, 1,270 observable feedback events and 1,129 template conversation records. The static snapshot version is 1.0.0. The dataset manifest identifies the six training, two validation and two development-test pools. Synthetic data is supplied for hackathon development, research and Vouchsafe integration testing. Do not present these profiles as real people or these outcomes as evidence of product effectiveness.

No extra personal data is needed. Any external training data must be lawfully available, have a declared licence and contain no unconsented sensitive personal information. Document external models and coding tools. An external dataset may support training but may not replace the supplied evaluation contract.

15 Product relevance and ownership

The intended output is a reusable policy for later assessment in Vouchsafe: explicit uncertainty, reciprocal eligibility, allocation across a changing pool and auditable decisions. Keep normalisation, modelling and allocation separate, and implement the portable JSON interface. Student submissions receive no production database access. docs/INTEGRATION.md defines an adapter boundary for authorised later evaluation without claiming that this synthetic schema is production-ready.

Students retain ownership of their submissions. Participation does not transfer intellectual property or grant a commercial licence to Vouchsafe or Romeo & Juliet. Any subsequent product licence, internship or development agreement must be voluntary and separate. Included starter code and synthetic data carry the repository's stated permissions; those permissions do not automatically apply to student work. No prizes, institutional endorsements or employment guarantees are promised by this technical specification.

Simulator success is a first engineering and research test. Real deployment requires validation on authorised inputs, current constraint checks and human review. Synthetic probabilities are not calibrated estimates for real members.